

Data Fetching and Loading States Differences with PWA Kit

PWA Kit follows a fetch-on-render model where hooks request data during component render and expose manual loading flags. Storefront Next follows fetch-then-render with route loaders that gather data before component rendering, enabling Suspense and streaming patterns more naturally. This moves data concerns out of presentation components and simplifies loading orchestration.

PWA Kit and Storefront Next handle loading states with different models. PWA Kit relies on manual conditional rendering, while Storefront Next uses React Suspense for declarative loading boundaries. PWA Kit's approach gives explicit control through React Query's `isLoading` flags and conditional rendering. Storefront Next's Suspense-based approach keeps components focused on rendering with data while boundaries handle loading automatically. That reduces boilerplate and enables streaming, but it requires understanding Suspense patterns and React 19's `use()` hook.

Paradigm

Aspect	PWA Kit	Storefront Next
Data Location	Inside components (hooks)	Outside components (loaders)
Discovery	Automatic (hook scanning)	Explicit (route exports)

Data Fetching

Feature	PWA Kit	Storefront Next
SSR Approach	React Query auto-discovery	Loader functions
Client Navigation	React Query refetch	Client loader

Contents

Paradigm

[Data Fetching](#)[Request Processing](#)[Loading States and Suspense](#)[Key Similarities](#)[Storefront Conversion Considerations](#)

[Try for free](#)

Caching	React Query cache	Manual / React Router
---------	-------------------	-----------------------

Request Processing

Feature	PWA Kit	Storefront Next
Pre-fetch Processing	Express middleware	React Router middleware
Shared State	Express <code>res.locals</code>	Router context
Request Access	Full Express <code>req / res</code>	Fetch API Request

Loading States and Suspense

Aspect	PWA Kit	Storefront Next
Approach	Manual conditional rendering	Declarative Suspense boundaries
Loading detection	<code>isLoading</code> flags from hooks	<code>use()</code> hook suspends on pending promises
Skeleton display	Explicit <code>if (isLoading)</code> checks	Automatic via Suspense fallback
Multiple regions	Manual state tracking per region	Multiple <code><Suspense></code> boundaries
Navigation transitions	<code>keepPreviousData</code> option	<code>getPageKey</code> + <code>shouldRevalidate</code>
Code splitting	<code>@loadable/component</code> fallback	React Router automatic splitting
Skeleton styling	Chakra <code><Skeleton></code> component	Tailwind <code>animate-pulse</code>
Streaming	Limited	Native promise streaming

[Try for free](#)

```
1 function ProductList() {
2   const { data: products, isLoading } = useProducts()
3
4   if (isLoading) {
5     return <ProductListSkeleton />;
6   }
7
8   return products.map((p) => <ProductTile product={p} />);
9 }
```

Storefront Next – declarative:

```
1 function ProductList({ loaderData }) {
2   const products = use(loaderData.products);
3   return products.map((p) => <ProductTile product={p} />);
4 }
5
6 export default createPage({
7   component: ProductList,
8   fallback: <ProductListSkeleton />,
9 });
```

Streaming Vs Waterfall

PWA Kit can experience client-side waterfalls when data-fetching hooks depend on each other:

```
1 // Sequential fetching
2 const { data: product } = useProduct({ id: product.id });
3 const { data: category } = useCategory({ id: product.categoryId });
```

Storefront Next enables streaming where data resolves as it's ready:

```
1 // Loader returns promises
2 export function loader({ params, context }) {
3   const productPromise = fetchProduct(params.productId);
4   const categoryPromise = productPromise.then((product) => fetchCategory(product.categoryId));
5
6   return {
7     product: productPromise,
8     category: categoryPromise, // Streams after product
9   };
10 }
```

Key Similarities

PWA Kit and Storefront Next have these data fetching capabilities.

- Server-side rendering.



Storefront Conversion Considerations

When converting your storefront from PWA Kit to Storefront Next, keep these tips in mind.

- Move data fetching to loaders: Extract `useQuery` calls into `loader` functions.
- Replace React Query: Use loader returns instead of hooks.
- Conditional rendering → Suspense: Replace `if (isLoading)` patterns with Suspense boundaries and the `createPage` factory.
- Update loading states: Replace `isLoading` conditionals with Suspense fallbacks.
- `keepPreviousData` → `shouldRevalidate`: Move “keep old data visible” logic from React Query options to router revalidation control.
- Loading spinners → skeleton fallbacks: Convert inline loading spinners to skeleton components in Suspense fallbacks.
- Multiple loading states → multiple boundaries: Split single-page loading into independent Suspense regions for granular streaming.
- Convert middleware: Express middleware becomes React Router middleware.
- Update request access: Use Fetch API `Request` instead of Express `req`.

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)



Try for free



Commerce Cloud

Lightning Design System

Einstein

Quip

Trailhead

Sample Apps

AppExchange

Blog

Salesforce Admins

Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc.
Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#)

[Terms of Service](#)

[Privacy Information](#)

[Responsible Disclosure](#)

[Trust](#)

[Contact](#)

[Use of Cookies](#)

[Cookie Preferences](#)



[Your Privacy Choices](#)

